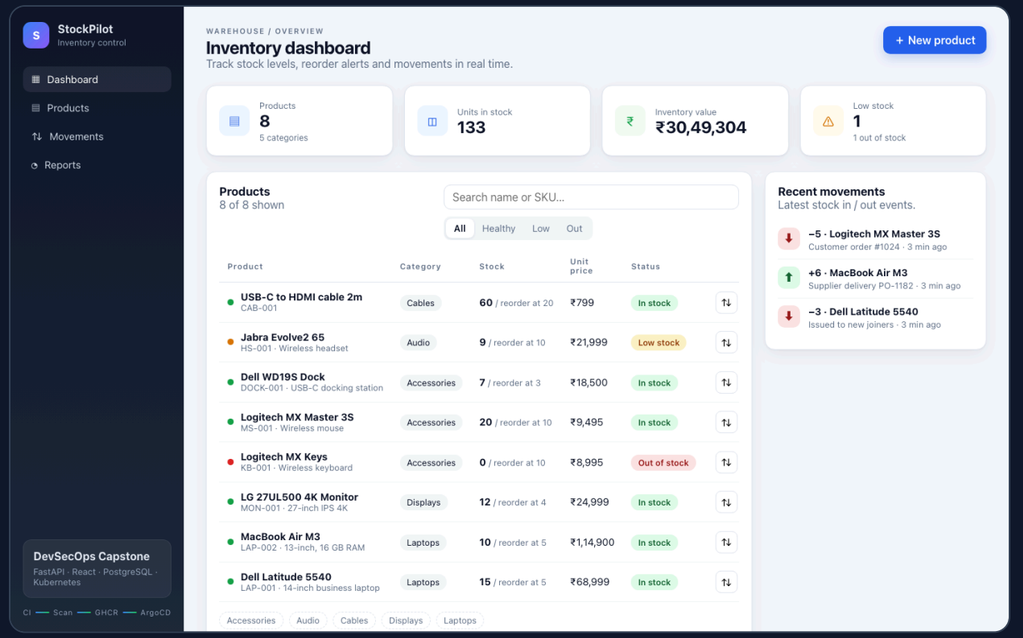
StockPilot — Inventory Management

DevSecOps Final Capstone · Gaurav Kumar

github.com/Gaurakumar-s/invenpilot-devsecops

FastAPI · React · PostgreSQL · Docker · GitHub Actions · Trivy · GHCR · Terraform · AWS EKS · Helm · ArgoCD · Prometheus · Grafana



StockPilot dashboard running on AWS EKS

Project Overview

PROBLEM

Small warehouses track stock in spreadsheets: no single source of truth and no reorder alerts, so items run out unnoticed.

SOLUTION — STOCKPILOT

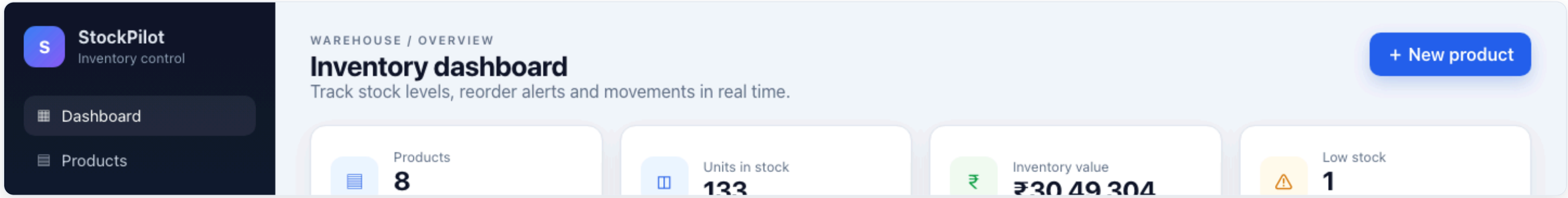
A web app that keeps a live product catalogue, records every stock movement, and flags what needs reordering.

FEATURES

- Products with SKU, category, quantity, reorder level, unit price
- Stock IN / OUT movements ledger
- Low-stock and out-of-stock alerts
- KPI dashboard: products, units, inventory value, low stock
- Responsive React UI with search and filters

REST API — 9 ENDPOINTS

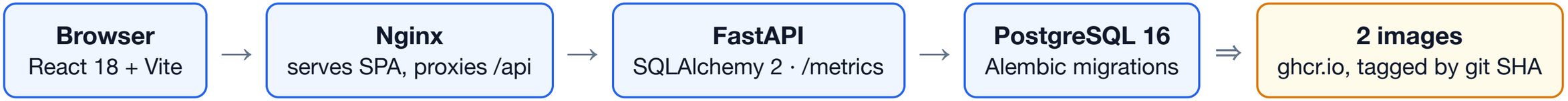
- GET / POST / PUT / DELETE `/api/products`
- GET `/api/products/stats` · GET `/api/products/low-stock`
- POST `/api/products/{id}/movements` · GET `/api/movements`
- Ops: `/health` · `/ready` · `/metrics` · `/docs`



Tech Stack

Application	DevOps / Platform
React 18 + Vite (frontend SPA)	Docker — multi-stage builds, non-root containers
FastAPI (Python REST API)	Docker Compose (local stack)
SQLAlchemy 2 + Alembic (2 migrations)	GitHub Actions (CI/CD) · Trivy (image scanning) · GHCR
PostgreSQL 16	Terraform — AWS VPC + EKS
pytest (20 tests) · ruff (lint)	Helm · ArgoCD · ingress-nginx · HPA
	Prometheus + Grafana (kube-prometheus-stack)

RUNTIME REQUEST PATH

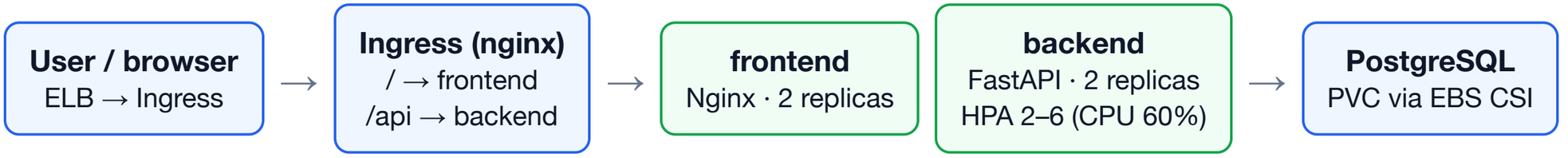


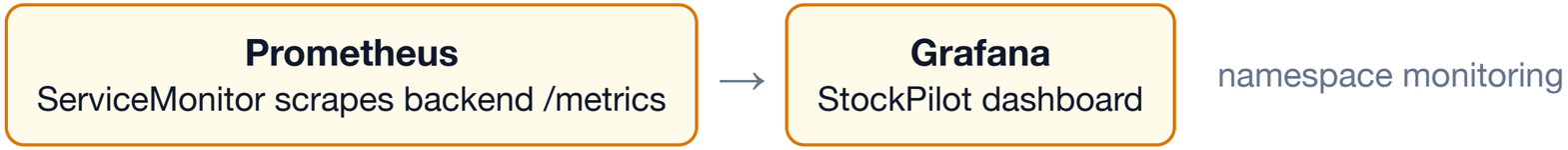
Architecture

DELIVERY PATH



AWS EKS — TERRAFORM-PROVISIONED VPC (2 PUBLIC + 2 PRIVATE SUBNETS, NAT) · NAMESPACE STOCKPILOT





namespace monitoring

CI/CD Pipeline — GitHub Actions

Triggers: push to main · pull request · manual (workflow_dispatch)

1 · backend-test
ruff lint + pytest (20 tests)
red tests fail the build

→

2 · frontend-build
npm ci + vite build

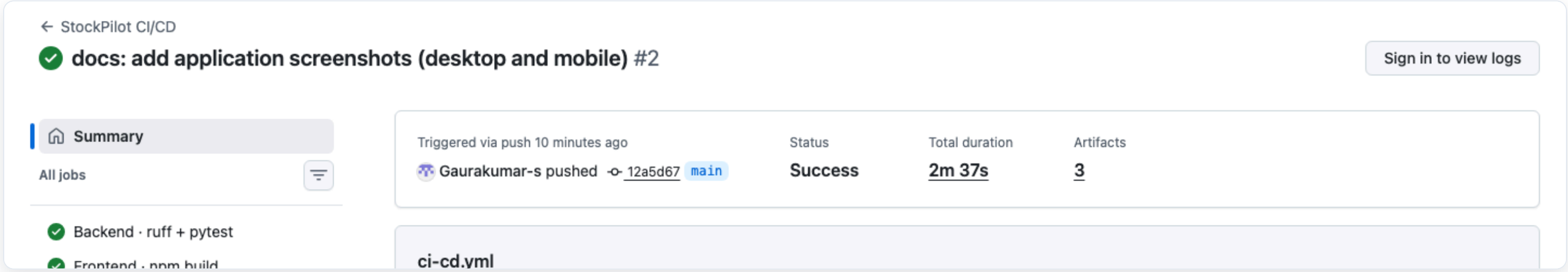
→

3 · build-scan-push
docker build both images
Trivy: HIGH/CRITICAL → exit 1
push to GHCR tagged :<git-sha>

→

4 · update-gitops
write SHA into helm values
commit → ArgoCD auto-syncs

- Jobs 1 and 2 run in parallel; job 3 needs both green; job 4 needs job 3 and runs only on main.
- Images: `ghcr.io/gaurakumar-s/stockpilot-backend:<git-sha>` and `…-frontend:<git-sha>` — never `latest`, so every pod traces back to one commit.
- Nothing is pushed unless tests and scans pass. The pipeline never runs kubectl: it changes Git, ArgoCD changes the cluster.



DevSecOps — Security Controls

TRIVY IMAGE SCANNING IN CI

- Scans OS packages + Python/npm libraries of both images
- Pipeline fails on HIGH / CRITICAL (ignore-unfixed)
- **Finding during build:** starlette 0.46 DoS (CVE-2025-62727) and old Alpine 3.21 packages (musl, zlib, libxml2)
- **Fix:** upgraded to starlette 1.8 / FastAPI 0.143 and nginx-unprivileged on Alpine 3.24

Scan status: 0 HIGH / 0 CRITICAL

OTHER CONTROLS

- Non-root containers: backend uid 10001, frontend uid 101
- securityContext drops ALL capabilities, no privilege escalation
- No secrets in git: .gitignore, terraform.tfvars.example, credentials only in the AWS CLI profile
- Least-privilege GITHUB_TOKEN (packages: write only in the push job)
- Immutable SHA tags; GitOps means no manual cluster access in CI

```
Report Summary
| ghcr.io/gaurakumar-s/stockpilot-backend:ab18a8dd424ce39edaff791952803e42e624fe75 |   debian   |       0       |   -   |
Legend:
- '0': Clean (no security findings detected)
  image-ref: ghcr.io/gaurakumar-s/stockpilot-frontend:ab18a8dd424ce39edaff791952803e42e624fe75
  INPUT_IMAGE_REF: ghcr.io/gaurakumar-s/stockpilot-frontend:ab18a8dd424ce39edaff791952803e42e624fe75
Running Trivy with options: trivy image ghcr.io/gaurakumar-s/stockpilot-frontend:ab18a8dd424ce39edaff791952803e42e624fe75
Report Summary
| ghcr.io/gaurakumar-s/stockpilot-frontend:ab18a8dd424ce39edaff791952803e42e624fe- | alpine |       0       |   -   |
```

Trivy output from the GitHub Actions run: both images clean (0 findings) before the push to GHCR

Infrastructure — Terraform on AWS

NETWORK (AP-SOUTHEAST-2)

- VPC 10.20.0.0/16 across 2 Availability Zones
- 2 public subnets — load balancer (ELB)
- 2 private subnets — EKS worker nodes
- Single NAT gateway for node egress

WORKFLOW

- terraform init → validate → plan → apply → destroy
- terraform.tfvars.example committed; no credentials in git
- ≈ $0.35 / hour — cluster destroyed after the demo

EKS CLUSTER

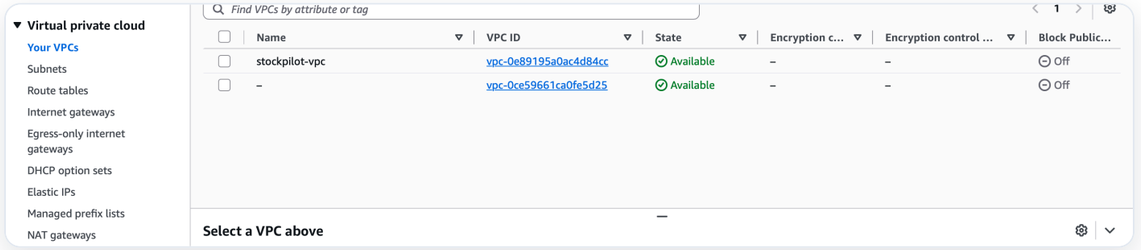
- EKS 1.36, managed node group
- m7i-flex.large × 2 (free-tier eligible, 2 vCPU / 8 GiB; min 2, max 2)
- Add-ons: coredns, kube-proxy, vpc-cni, aws-ebs-csi-driver (EBS policy on the node role) — backs the Postgres PVC

VPC 10.20.0.0/16

public ×2
ELB + NAT

private ×2
EKS nodes

AZ-a and AZ-b each hold one public and one private subnet. Region is fixed by the account's organization policy.



AWS console: stockpilot-vpc in ap-southeast-2

Kubernetes · Helm · ArgoCD

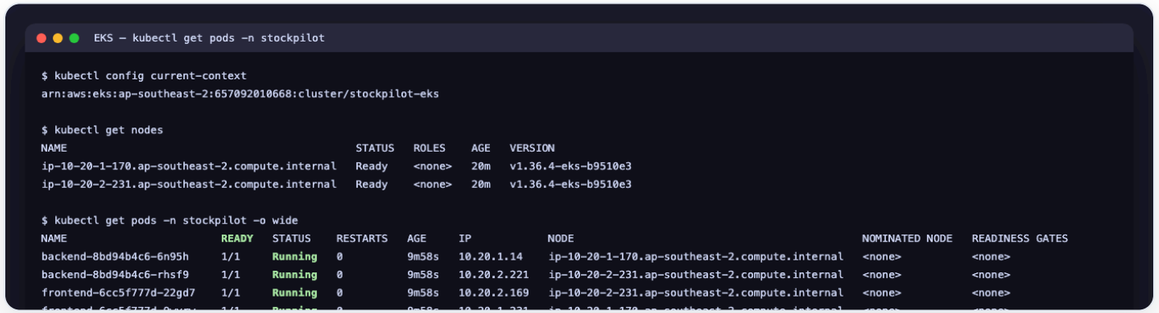
HELM CHART — HELM/STOCKPILOT

- Deployments: backend ×2, frontend ×2, postgres ×1 + PVC
- ClusterIP Services; Secret for DB credentials
- Ingress (nginx class): / → frontend, /api → backend
- HPA on backend CPU 60 %: 2 → 6 replicas
- ServiceMonitor for Prometheus; liveness + readiness probes
- values-minikube.yaml / values-eks.yaml

ARGOCD — GITOPS

- Application points at the chart in the repo (main)
- Automated sync + prune + selfHeal
- Git is the source of truth: drift is reverted, deletions pruned

```
$ kubectl get pods,svc,ingress -n stockpilot
$ helm list -n stockpilot
$ kubectl get hpa -n stockpilot
$ argocd app get stockpilot
```





Observability — Prometheus + Grafana

METRICS PIPELINE

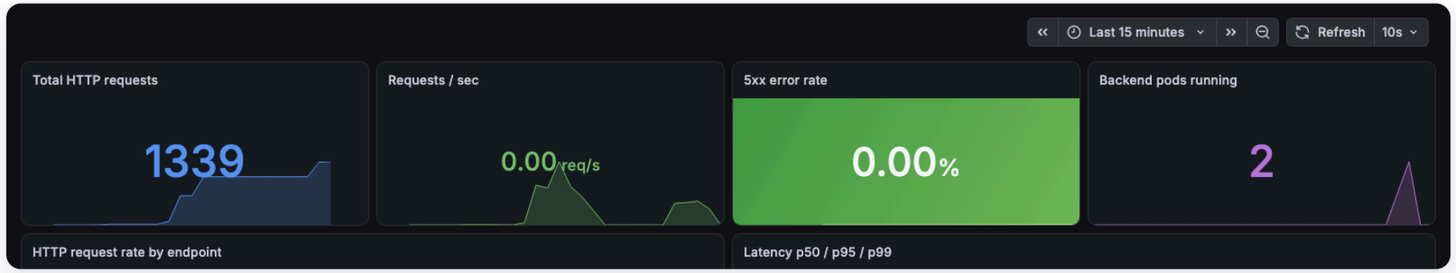
- Backend exposes /metrics via prometheus-fastapi-instrumentator
- http_requests_total, http_request_duration_seconds
- kube-prometheus-stack in namespace monitoring
- ServiceMonitor (release=kube-prometheus-stack) → Targets page: backend UP
- load-test.sh generates traffic for the panels

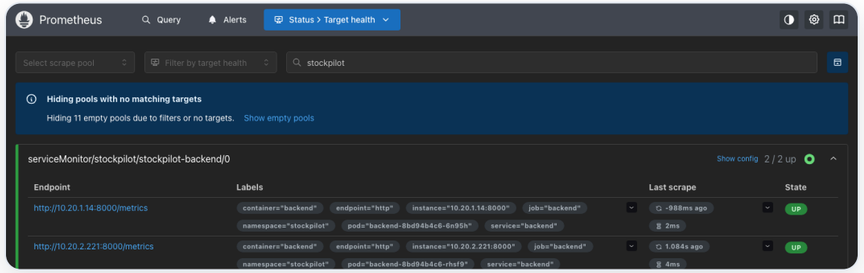
backend /metrics → **ServiceMonitor** → **Prometheus** → **Grafana**

GRAFANA — "STOCKPILOT — APPLICATION METRICS"

Requests / sec	p50 / p95 / p99 latency
5xx error rate	Responses by status code
Backend CPU (HPA input)	Pods running

Six panels on one dashboard: traffic, latency, errors, and the CPU signal that drives the HPA.





Key Learnings & Demo Agenda

KEY LEARNINGS

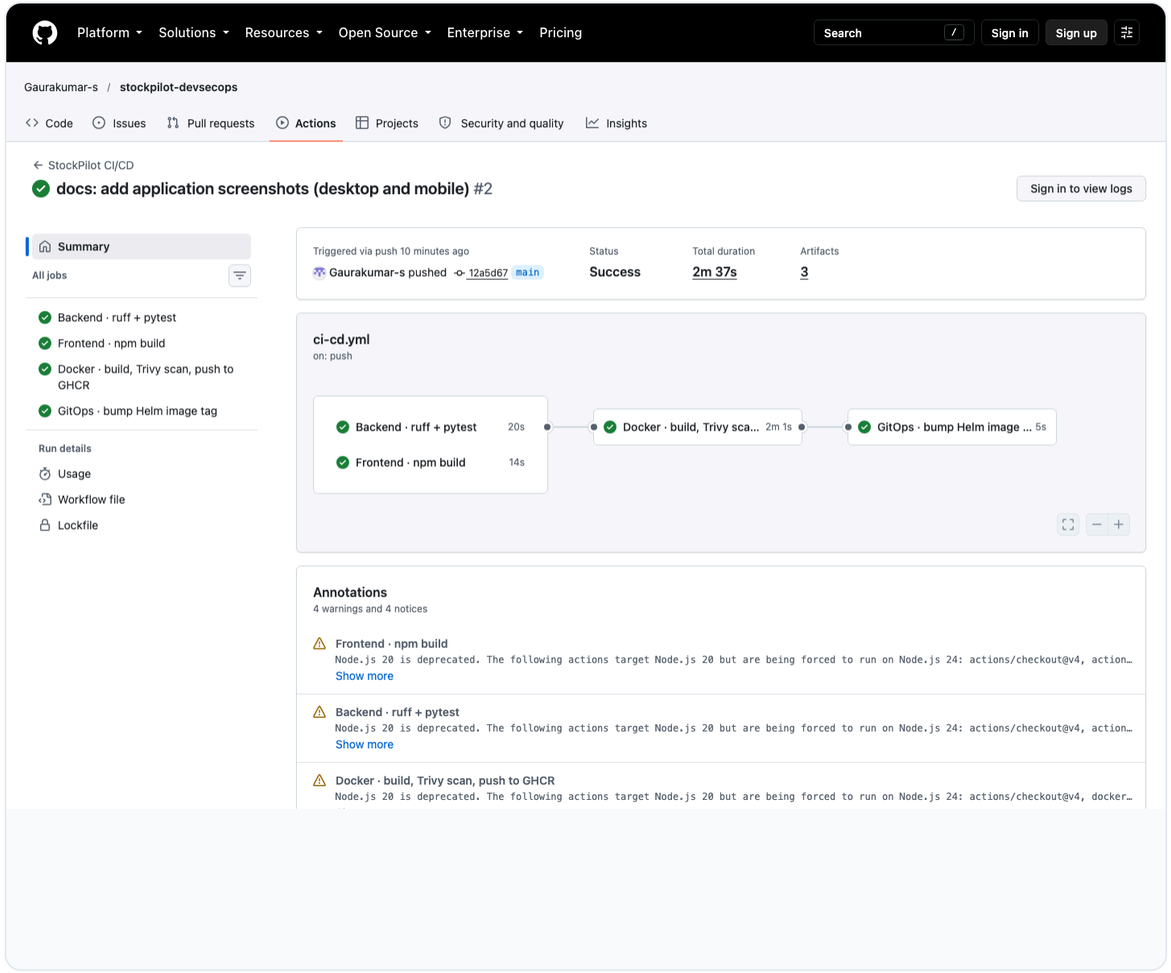
- Quality gates before promotion: tests → scan → registry
- Immutable SHA tags give end-to-end traceability
- GitOps removes manual kubectl deploys
- PVCs on EKS need the EBS CSI driver
- Cloud accounts have real constraints: region policy, free-tier instance types, quotas
- Always destroy cloud infrastructure when done

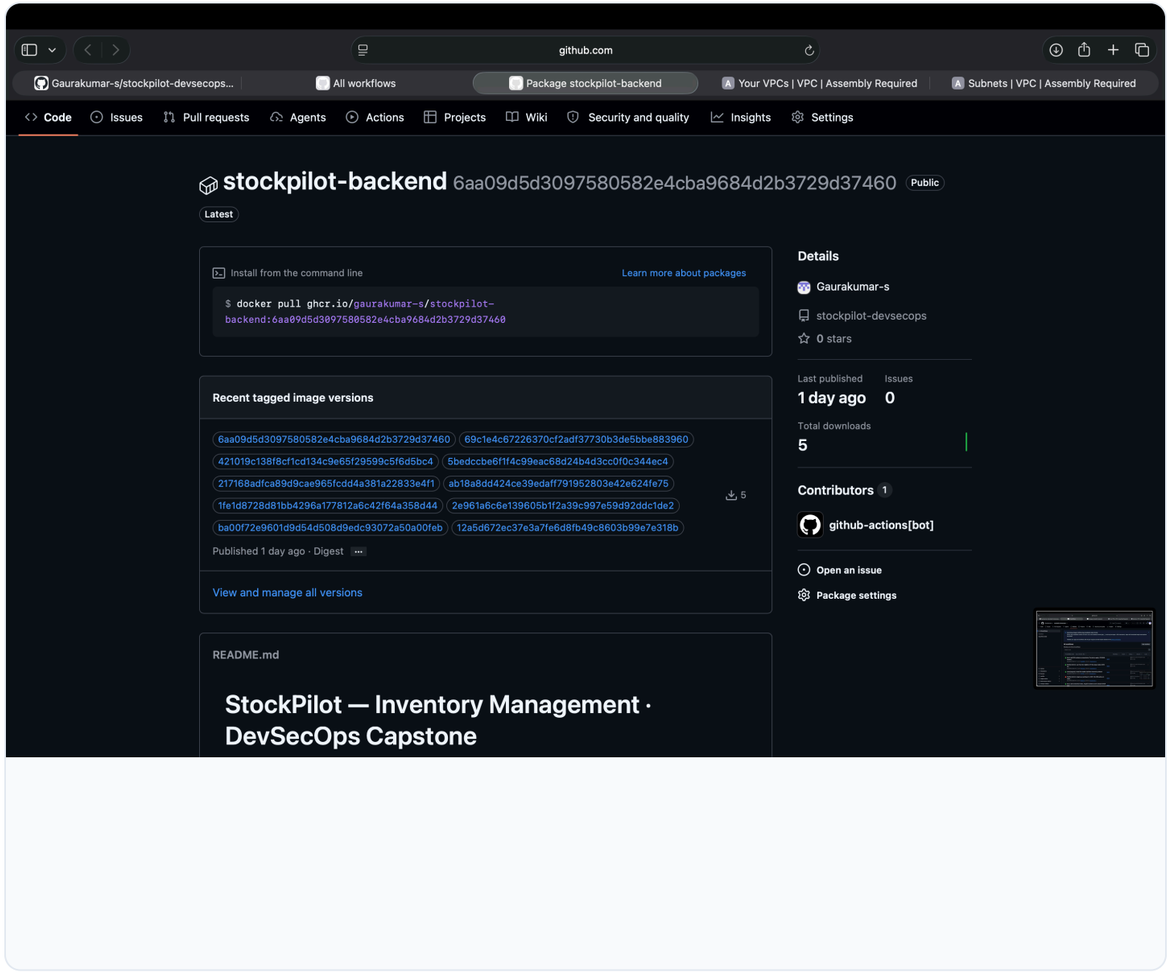
LIVE DEMO AGENDA

1. Codebase tour
2. Push a commit, watch GitHub Actions
3. GHCR: new SHA-tagged images
4. terraform plan + AWS console
5. ArgoCD: synced application
6. kubectl pods / services / ingress, app via load balancer
7. Prometheus targets
8. Grafana dashboard

Thank you · github.com/Gaurakumar-s/invenpilot-devsecops

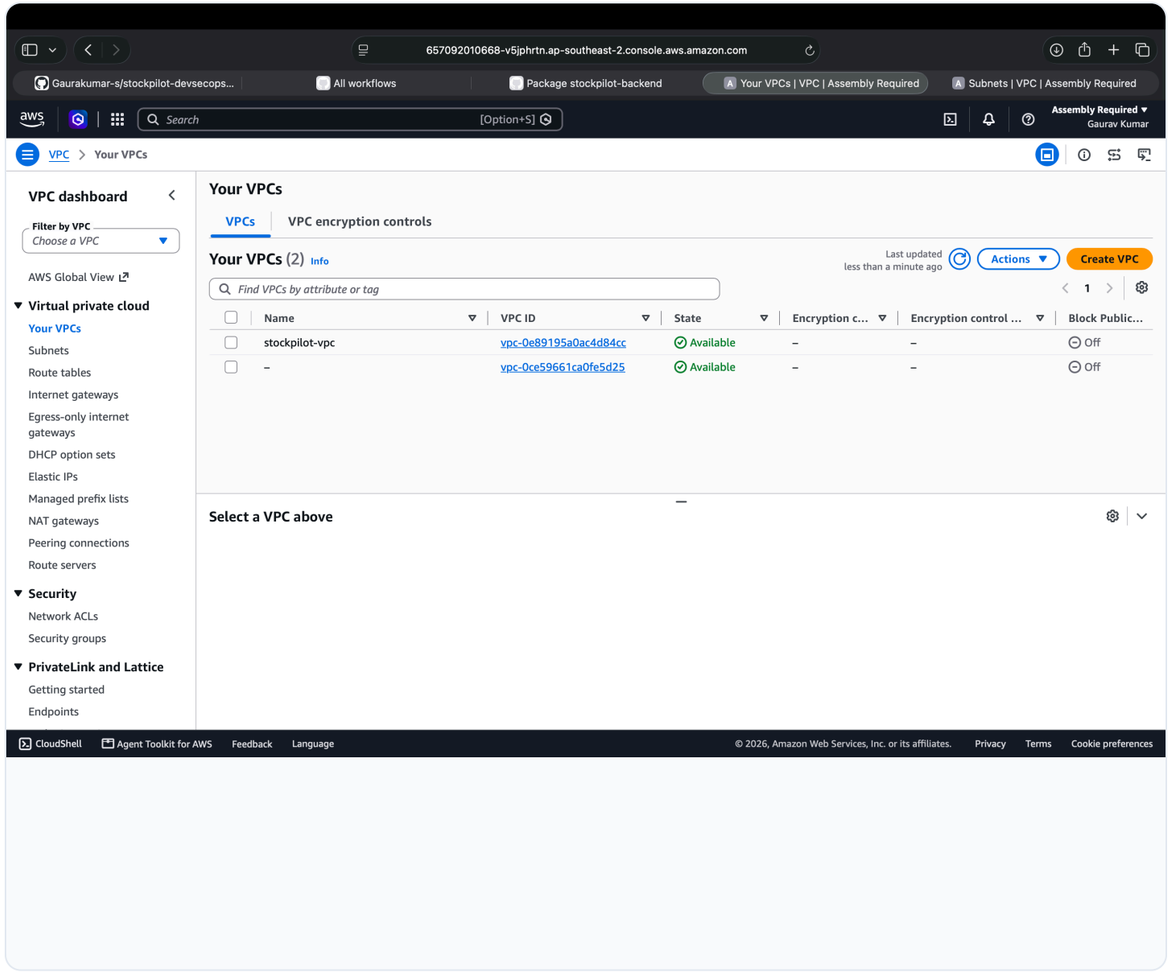
Evidence — CI/CD run and registry

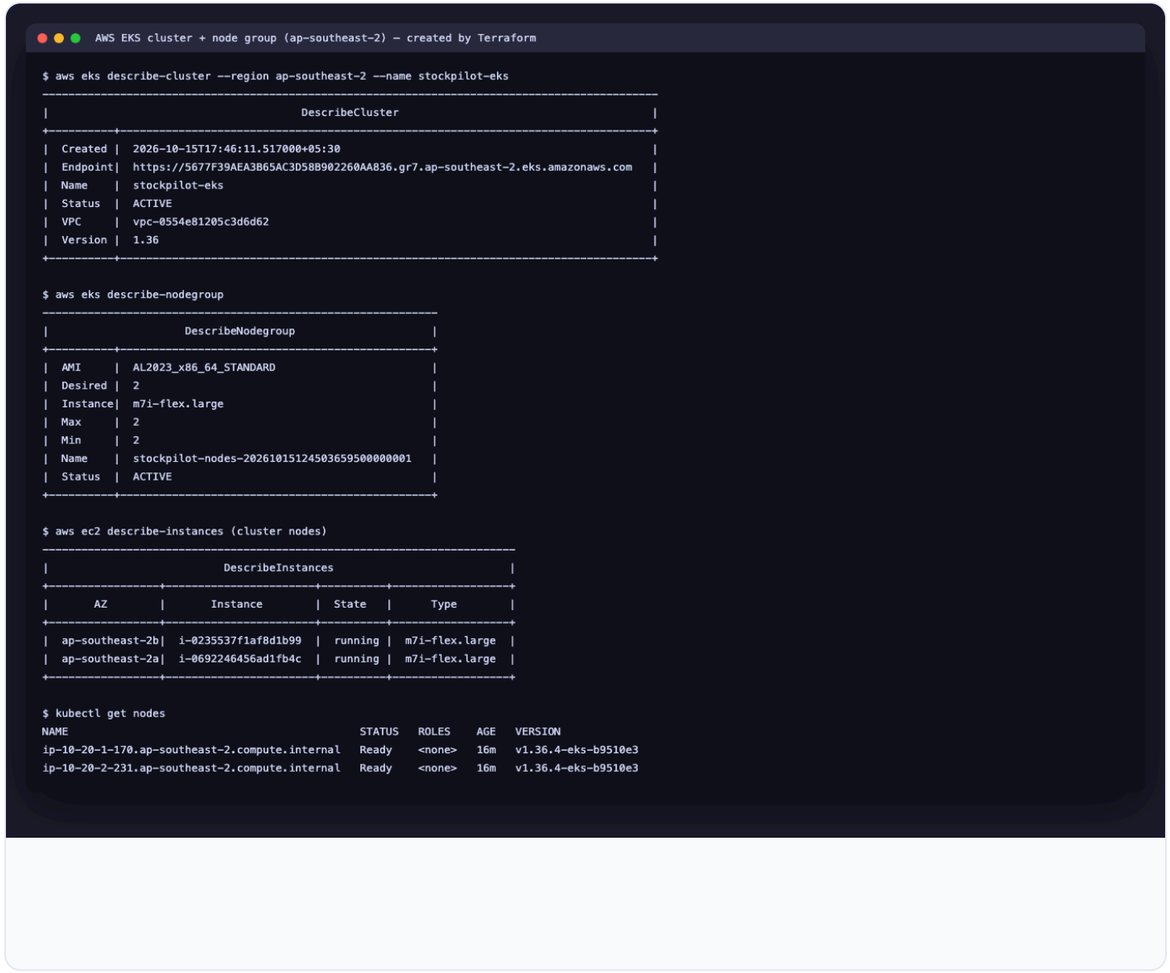




Left: one push, four green jobs (test → build → Trivy scan + push → GitOps bump). Right: GHCR package tagged only with commit SHAs.

Evidence — Terraform on AWS





Left: the VPC in the AWS console (ap-southeast-2). Right: EKS cluster, managed node group and EC2 nodes created by `terraform apply`.

Evidence — Kubernetes, Helm and ArgoCD

EKS — kubectl get pods -n stockpilot

```
$ kubectl config current-context
arn:aws:eks:ap-southeast-2:657092010668:cluster/stockpilot-eks

$ kubectl get nodes
NAME                                             STATUS   ROLES    AGE   VERSION
ip-10-20-1-170.ap-southeast-2.compute.internal   Ready    <none>   20m   v1.36.4-eks-b9510e3
ip-10-20-2-231.ap-southeast-2.compute.internal   Ready    <none>   20m   v1.36.4-eks-b9510e3

$ kubectl get pods -n stockpilot -o wide
NAME                        READY   STATUS    RESTARTS   AGE     IP            NODE                                             NOMINATED NODE   READINESS GATES
backend-8bd94b4c6-6n95h     1/1     Running   0          9m58s   10.20.1.14    ip-10-20-1-170.ap-southeast-2.compute.internal   <none>           <none>
backend-8bd94b4c6-rhsf9     1/1     Running   0          9m58s   10.20.2.221   ip-10-20-2-231.ap-southeast-2.compute.internal   <none>           <none>
frontend-6cc5f777d-22gd7    1/1     Running   0          9m58s   10.20.2.169   ip-10-20-2-231.ap-southeast-2.compute.internal   <none>           <none>
frontend-6cc5f777d-9wvrw    1/1     Running   0          9m58s   10.20.1.231   ip-10-20-1-170.ap-southeast-2.compute.internal   <none>           <none>
postgres-574b64fdb7-rv6x6   1/1     Running   0          9m58s   10.20.2.4     ip-10-20-2-231.ap-southeast-2.compute.internal   <none>           <none>

$ kubectl get deploy -n stockpilot
NAME       READY   UP-TO-DATE   AVAILABLE   AGE
backend    2/2     2            2           10m
frontend   2/2     2            2           10m
postgres   1/1     1            1           10m
```

ArgoCD on EKS — argocd app get stockpilot (Synced / Healthy)

```
$ kubectl config current-context
arn:aws:eks:ap-southeast-2:657092010668:cluster/stockpilot-eks

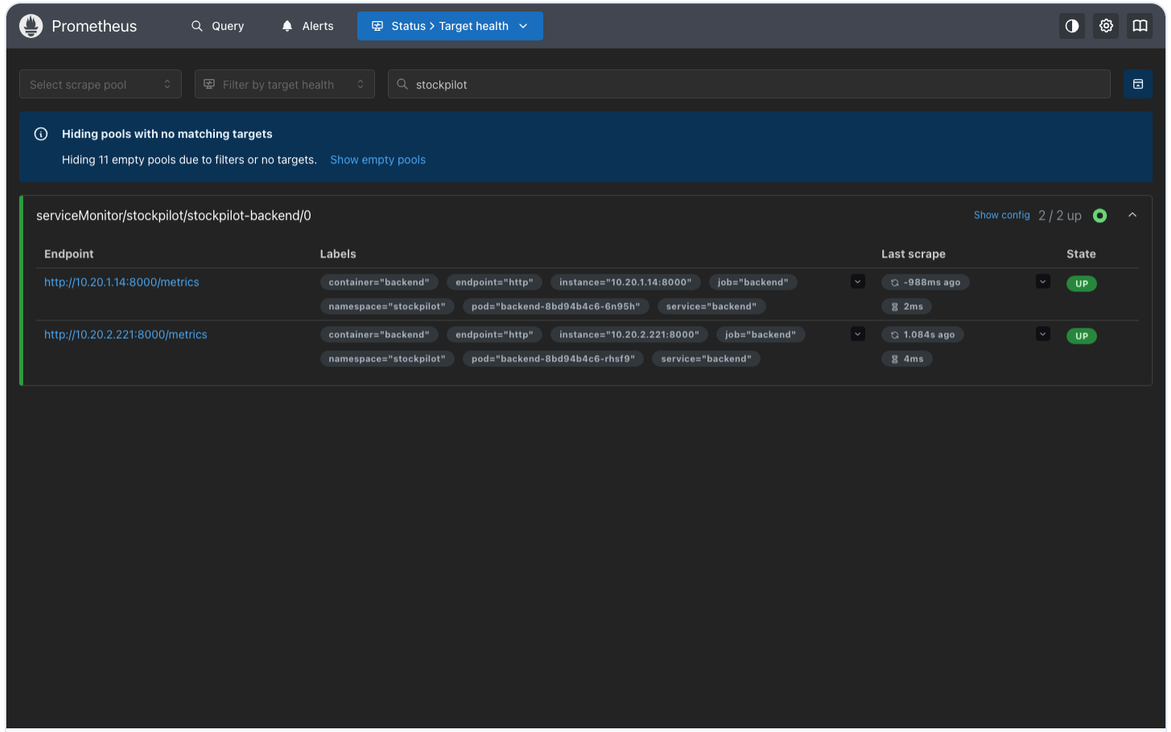
$ argocd app list
NAME              CLUSTER                         NAMESPACE   PROJECT  STATUS  HEALTH   SYNCPOLICY  CONDITIONS  REPO
PATH              TARGET
argocd/stockpilot  https://kubernetes.default.svc  stockpilot  default  Synced  Healthy  Auto-Prune  <none>      https://github.com/Gaurakumar-s/stockpilot-devsecops.g
it  helm/stockpilot  main

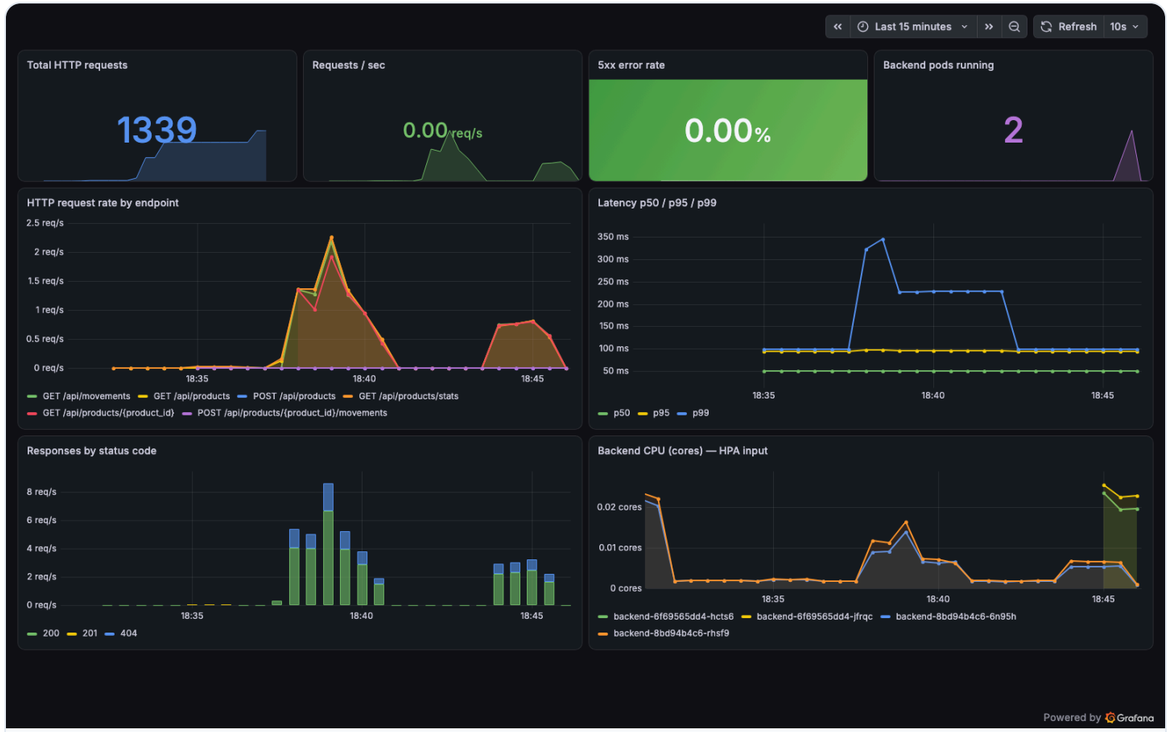
$ argocd app get stockpilot
Name:               argocd/stockpilot
Project:            default
Server:             https://kubernetes.default.svc
Namespace:          stockpilot
URL:                https://127.0.0.1:8443/applications/stockpilot
Source:
- Repo:             https://github.com/Gaurakumar-s/stockpilot-devsecops.git
  Target:           main
  Path:             helm/stockpilot
  Helm Values:      values.yaml,values-eks.yaml
SyncWindow:         Sync Allowed
Sync Policy:        Automated (Prune)
Sync Status:        Synced to main (6498e07)
Health Status:      Healthy

GROUP                KIND                    NAMESPACE   NAME               STATUS  HEALTH   HOOK  MESSAGE
apps                 Deployment              stockpilot  frontend           Synced  Healthy        deployment.apps/frontend serverside-applied
                     PersistentVolumeClaim   stockpilot  postgres-data      Synced  Healthy
                     Secret                  stockpilot  stockpilot-db      Synced
                     Service                 stockpilot  backend            Synced  Healthy
                     Service                 stockpilot  frontend           Synced  Healthy
                     Service                 stockpilot  postgres           Synced  Healthy
apps                 Deployment              stockpilot  backend            Synced  Healthy
apps                 Deployment              stockpilot  postgres           Synced  Healthy
autoscaling          HorizontalPodAutoscaler stockpilot  backend            Synced  Healthy
monitoring.coreos.com ServiceMonitor         stockpilot  stockpilot-backend Synced
networking.k8s.io    Ingress                 stockpilot  stockpilot         Synced  Healthy
```

Left: 2 backend, 2 frontend and 1 PostgreSQL pod Running on EKS. Right: ArgoCD application Synced / Healthy with automated sync, prune and self-heal.

Evidence — Prometheus and Grafana





Left: Prometheus Targets page, both backend pods UP via the ServiceMonitor. Right: the application dashboard with live request rate, latency, error rate and CPU.